

phorce SDK — API 레퍼런스

이 문서는 SDK가 여러분에게 열어 준 **모든 창구의 사전**입니다. 앞부분(시스템 지도와 공통 규칙)만 읽고, 그 다음부터는 **여러분이 쓰는 언어의 탭 하나만** 골라 보면 됩니다 — Python 만 쓸 거라면 Python 탭만 보면 충분합니다.

1. 여러분이 서 있는 곳 — 무엇을 만들고, 무엇을 맡기는가

phorce 시스템에서 Jetson 은 **로봇의 두뇌이자 여러분의 개발 컴퓨터**입니다. 역할 분담은 이렇게 정해져 있습니다:

	누가	무엇을
판단	여러분의 코드	로봇의 상태를 보고 "언제, 몇 번 모션을 재생할지" 결정 — AI 정책, 제어 로직, 데모 시나리오 전부 여기
실행	우리가 만든 시스템	1 초에 1000 번(1kHz) 로봇과 통신하며 모션을 실제로 재생하고, 매 순간 안전을 검사

제약 조건 — 먼저 알고 시작하세요.

- ① **관절을 직접 실시간으로 조종하는 길은 막혀 있습니다.** "3 번 관절을 0.5 초 동안 30 도로" 같은 명령을 보내는 API 는 없습니다. 이것은 안전 설계입니다 — 빠른 제어 루프는 검증된 시스템만 만질 수 있습니다.
- ② 로봇을 움직이는 방법은 **모션 번호(1~50) 전송, 단 하나**입니다. 로봇에 미리 담겨 있는 모션(동작 파일)을 번호로 골라 재생하는 방식입니다 — 노래방에서 곡 번호를 입력하는 것과 같습니다.
- ③ 대신 **관측은 활짝 열려 있습니다.** 모든 관절의 위치·속도·전류·온도가 1 초에 1000 번 여러분에게 옵니다. 이 데이터로 판단하는 코드가 여러분의 작품입니다.

2. 시스템 지도

여러분의 명령이 로봇까지 내려가는 길(왼쪽 ▼)과, 로봇의 상태가 여러분에게 올라오는 길(오른쪽 ▲)입니다. 초록 =

여러분의 영역, 보라 = SDK, 갈색 = 로봇.

여러분의 코드 — AI 정책 · 제어 로직 · 시나리오

어떤 언어든: 터미널(CLI) · Python · C++ · 화면(GUI) · ROS 2 직접

▼ "3 번 재생해" (모션 번호) ▲ 관절 상태 1 초에 1000 번 (피드백)

SDK 공개 창구 (누구나 같은 규칙)

재생 액션 PlayMotionSequence 하나 + 관측 토픽 4 개 + 목록 서비스 1 개



게이트웨이 — 요청 검증 · 안전 검사 · 1kHz 통신 (SDK 가 운영 — 여러분이 만지지 않음)

▼ ▲ EtherCAT 케이블 (1kHz)

로봇 — pcm(모션 재생과 안전의 최종 결정권자) → phact × 6(관절 모터)

이 그림이 말해 주는 것: 어느 문(언어)으로 들어가도 전부 가운데의 **같은 공개 창구**로 모입니다. 그래서 CLI 로 되는 일은 Python 으로도, C++ 로도 똑같이 됩니다. 그리고 여러분의 코드에 실수가 있어도 게이트웨이와 pcm 이 이상한 요청을 거절하므로 로봇은 다치지 않습니다.

3. 자주 나오는 낱말 6 개 — 먼저 알아 두면 다 쉬워집니다

낱말	뜻
토픽 (topic)	계속 흘러나오는 방송 채널. 구독하면 새 데이터가 올 때마다 받습니다 (예: 피드백)
액션 (action)	"해 줘" 하고 부탁하면 → 진행 소식을 받다가 → 끝나면 결과를 받는 통신. 모션 재생이 액션입니다
서비스 (service)	질문 하나, 답 하나 (예: "모션 목록 줘")

네임스페이스 이름 앞에 붙는 주소. 이 행사 구성에서는 전부 최상위(빈 주소)라 신경 쓸 일 없습니다

블로킹 / 논블로킹 끝날 때까지 기다리는 호출 / 바로 돌아오고 나중에 결과를 받는 호출

콜백 (callback) "데이터가 오면 이 함수를 불러 줘" 하고 맡겨 두는 함수

4. 공통 규칙 — 어떤 언어를 쓰든 똑같습니다

4-1. 대상(target) — 기본값 그대로 두면 실물 로봇입니다

CLI·Python·C++ 모두 **아무 설정 없이 실행하면 여러분의 실물 로봇에 연결됩니다** (기본 대상 `robot`). 이 행사에서는 이 기본값만 쓰면 되고, 대상을 바꿀 일이 없습니다.

참고: 여기저기서 "**시뮬레이터**"라는 선택지(sim:이름 표기, GUI의 대상 칸 등)를 보게 될 텐데, 이 행사에서는 **다루지 않습니다**. 여러분에게는 실물 로봇이 있으니까요. 시뮬레이터 대상을 고르면 로봇이 안 움직일 뿐입니다 — 실수로 실물이 움직이는 방향의 혼동은 없습니다.

4-2. 모션 번호

- 재생할 수 있는 번호는 **1부터 50**. 어떤 번호가 실제로 채워져 있는지는 로봇이 알고 있습니다 — `phorce list`로 물어보세요.
- **0은 금지**입니다. "모션 없음"을 뜻하는 예약 번호라서 재생 요청에 넣으면 거절됩니다.
- **한 번에 하나만** 보냅니다. 두 개를 이어 보내고 싶으면, 첫 번째의 **결과를 받은 뒤** 두 번째를 보내세요.

4-3. 거절 — 세 가지 경우, 세 가지 대응

요청이 거절되면 번호와 함께 이유가 옵니다. 딱 세 가지만 기억하세요:

코드	이름	무슨 상황	여러분이 할 일
5	<code>QUEUE_FULL</code>	다른 모션이 아직 재생 중	기다렸다가 다시 보내면 됩니다 . 재시도해도 되는 유일한 경우

12	<code>NOT_READY_FOR_MOTION</code>	로봇이 모션 받을 준비가 안 됨	기다려도 안 풀립니다 — 로봇의 영점 버튼(1 번) 을 0.6 초 누르면 약 3 초 뒤 준비됩니다
13	<code>RECOVERY_REQUIRED</code>	직전 동작이 실패해서 복구 필요	2 번 버튼(파킹) → 영점 버튼 순서로 사람이 복구합니다

가장 흔한 버그: 12·13 까지 "바쁘니까 재시도"로 처리하면 프로그램이 **영원히 재시도만 합니다**. "바쁨(busy)"은 코드 5 하나뿐입니다. 전체 거절 코드 목록은 ROS 2 탭에 있습니다.

표의 이름은 줄인 표기입니다 — 정식 상수와 Python `.reason` 문자열에는 `REJECT_` 접두사가 붙습니다 (예: `REJECT_QUEUE_FULL`). 코드에서 비교할 때는 숫자(`.code`)가 안전합니다.

4-4. 성공 판정 — `ok` 하나만 보면 됩니다

"성공"은 생각보다 간단한 개념입니다. 요청이 끝났다고 다 성공이 아니라, **로봇이 이 요청을 진짜 받아서 · 끝까지 재생했고 · 지금 멈춰 있고 · 복구가 필요 없는 상태까지** 전부 확인돼야 성공입니다. 이 판정을 SDK 가 `ok` 하나로 합쳐 줍니다 — 직접 조립하지 마세요.

4-5. 멈추고 싶을 때

즉시 정지는 물리 E-Stop 버튼뿐입니다. 코드의 `cancel()` 은 "아직 시작 안 한 것을 취소해 달라"는 부탁일 뿐이고, 이미 시작된 동작은 끝까지 갑니다. 실물 로봇은 재생 중 취소 기능 자체가 없습니다.

4-6. 데이터를 믿어도 되는가 — 신선도

- 관절 데이터가 신선하다는 **유일한 증거는 `valid` 필드**입니다. "낡지 않았으니(`stale` 아님) 신선하겠지"는 틀립니다 — 데이터가 아예 안 온 관절은 둘 다 `false` 라서요.
- 모션 상태 정보는 나이가 **1500ms 에 도달하면**(`age_ms ≥ 1500`) "낡음(STALE)"으로 판정됩니다. "로봇이 지금 움직이고 있나?"는 이 값 말고 **1kHz 피드백**으로 보세요.

5. 이제 문을 고르세요

아래 다섯 부분은 각각 독립된 사용 설명서입니다. **화면으로 보고 있다면 아래 버튼으로 원하는 것만 골라 볼 수 있습니다** (인쇄물에는 전부 순서대로 실려 있습니다).

문	이럴 때 고르세요
CLI — 터미널 명령 <code>phorce</code>	일단 빨리 해 보고 싶다 / 셸 스크립트로 자동화하고 싶다
Python	제일 편한 프로그래밍 경로 — 대부분의 참가자에게 추천
C++	ROS 2 노드에 통합하거나 성능이 필요할 때
ROS 2 직접	SDK 도우미 없이 통신 규약을 직접 쓰고 싶을 때 · 다른 언어를 쓸 때
GUI — 화면 <code>phorce-console</code>	코드 없이 그래프를 보고 버튼으로 재생하고 싶을 때

CLI — 터미널 명령 `phorce`

로그인만 하면 바로 쓸 수 있습니다. "서버를 찾지 못했습니다"가 나오면 로봇 스택이 안 켜진 것입니다 — ① 퀵 가이드 3-2 절(시동 걸기 — 터미널 2 개)부터. 명령은 네 개뿐입니다:

명령	하는 일	기다리는 시간(기본)
<code>phorce doctor</code>	건강검진 — 연결이 되는지, 서버가 살아 있는지, 모션 목록이 준비됐는지 검사	2 초
<code>phorce list</code>	재생할 수 있는 모션 번호와 이름 목록	5 초
<code>phorce play</code> 번호	그 번호 모션을 재생하고 끝날 때까지 기다림	30 초
<code>phorce status</code>	로봇의 모션 상태 한 장 (보기만 — 아무 명령도 안 보냄)	2 초

모든 명령에 붙일 수 있는 옵션:

옵션	뜻
<code>--timeout</code> 초	기다리는 시간 바꾸기
<code>--json</code>	사람용 출력 대신 기계용 JSON 한 줄 — 스크립트에서 파싱할 때
<code>--target / --namespace / --domain-</code> <code>id</code>	대상·주소를 직접 지정 — 이 행사에서는 기본값(실물 로봇)이면 되므로 쓸 일 없음

끝나고 남기는 번호(exit code) — 스크립트에서 \$? 로 확인합니다:

번호	뜻
0	성공
1	거절되거나 실패 (화면의 안내문을 읽으세요 — 할 일이 적혀 있습니다)
2	명령을 잘못 씀
3	연결할 서버가 없음 (스택이 안 떠 있거나 target 오타)
4	시간 초과 (재생 취소를 요청하고 끝나지만, 로봇 동작은 계속될 수 있음)
5	바쁨 — 요청은 버려졌으니 기다렸다 다시
130	Ctrl+C 로 직접 끊음



Python — import phorce

가장 짧은 완전한 프로그램부터 보세요:

```
import phorce
import sys

def main():
    robot = phorce.connect(target="robot", *, timeout=10.0, namespace=None, domain_id=None, require_server=True)
    doctor = phorce.doctor(target="robot", *, timeout=2.0, namespace=None, domain_id=None)
    print("ok") if result.ok else result.detail

if __name__ == "__main__":
    main()
```

연결 만들기

```
phorce.connect(target="robot", *, timeout=10.0, namespace=None, domain_id=None, require_server=True) → Robot
```

로봇과의 연결을 만듭니다. timeout 안에 서버가 안 보이면 `PhorceUnavailable` 예외가 납니다. 여러분이 이미 ROS 2 코드를 쓰고 있어도 안전합니다 — connect 는 자기만의 독립된 통신 문맥을 만들어서 여러분의 `rclpy` 초기화와 충돌하지 않습니다.

```
phorce.doctor(target="robot", *, timeout=2.0, namespace=None, domain_id=None) → DoctorReport
```

건강검진만 합니다 — 모션은 보내지 않고, 서버가 없어도 동작합니다. `report.ok` 와 `report.issues`(문제 목록)를 보세요.

Robot — 연결의 중심

메서드	하는 일
<code>play(*ids, stop_on_error=True, timeout=None) → PlayResult</code>	블로킹 재생. 번호는 괄호 안에 그냥 넣으세요: <code>robot.play(3).ms_id=3</code> 처럼 이름 붙여 넣는 형식은 없습니다. timeout 을 주면 그 시간 넘겼을 때 취소를 요청하고 <code>TimeoutError</code> (로봇 동작은 계속될 수 있음)
<code>play_async(*ids,</code>	논블로킹 재생 — 바로 돌아오고, 진행 소식은 <code>on_feedback</code> 콜백으로

stop_on_error=True,

on_feedback=None) → PlayHandle

status(timeout=2.0) → Status 상태 한 장 읽기 (아무 명령도 안 보냄)

motions() → Catalog 모션 목록. `for m in robot.motions():` 처럼 바로 순회

doctor() / close() 진단 / 연결 닫기 (with 문이 자동으로 해 줌)

PlayResult — 재생이 끝나면 받는 것

속성	뜻
<code>ok</code>	성공 여부. 괄호 없이 <code>result.ok</code> 입니다 (<code>result.ok()</code> 라고 쓰면 에러). 공통 규칙 4-4 의 간단한 판정을 다 합친 값
<code>status_name</code>	"SUCCEEDED" 또는 "CANCELED" 둘뿐 . 거절·중단은 결과로 돌아오지 않고 아래의 예외로 던져집니다 — <code>status_name == "REJECTED"</code> 같은 분기는 절대 실행되지 않는 죽은 코드가 됩니다
<code>detail</code>	사람이 읽는 한국어 설명 — 문제가 있으면 할 일 이 적혀 있으니 사용자에게 그대로 보여 주세요
<code>completed_count</code>	끝까지 재생된 개수 (0 또는 1)
그 외	<code>boot_id</code> · <code>request_id</code> · <code>primary_state</code> · <code>physical_idle</code> · <code>recovery_required</code> 등 로봇 쪽 문맥 — 궁금할 때만 보면 됩니다 (ROS 2 탭에 설명)

예외 — 오류는 타입으로 구분합니다

<code>RobotError</code>	로봇 쪽 예외 (모든 예외의 기본)
<code>RobotClosedError</code>	연결이 닫혔을 때
<code>RobotRejectedError</code>	거절된 모션 (예외로 던져집니다) (이 예외는 <code>RobotError</code> 의 하위 클래스입니다)
<code>RobotBusyError</code>	로봇이 바쁠 때 (예외로 던져집니다)

```

not authorized # 시간이 꽤나 걸린다
#
result = robot.play()
#
your chance not finished
#
time.sleep(1) # 뒤를 한 번 돌아서 내가 움직이지
#
your chance not interrupted as
#
while is_detail: # 움직이지 않을 때는 while True = while 1
#
    your chance not interrupted
#
print("전공자 이름 : ", phorce doctor 0)

```

PlayHandle — play_async 가 돌려주는 손잡이

멤버	뜻
----	---

`wait(timeout=None) → PlayResult` 끝나기를 기다림. timeout 넘으면 `TimeoutError` — 재생은 계속됩니다

cancel() 취소 부탁 (공통 규칙 4-5 — E-Stop 아님)

done / last_feedback 끝났는지 / 마지막으로 받은 진행 소식

Status — 상태 한 장

`st = robot.status()` 로 받습니다. 제일 유용한 것은 `st.state_name` 입니다:

state_name	뜻
------------	---

IDLE 진짜 쉬는 중 — 새 모션을 보내기 좋은 때

EXECUTING 등 재생 진행 중 (지금 보내면 바쁨 거절)

RECOVERY_REQUIRED 사람 복구 필요 (거절 13 과 같은 상황)

STALE 정보가 낡음 (1.5 초 이상) — 판단에 쓰지 마세요

`CONTRACT_INACTIVE` / `UNKNOWN` 로봇 창구가 안 열렸거나 문맥 미확보 — 이때 다른 필드로 "쉬는 중"을 추측하지 마세요

진행 소식 Feedback (on_feedback 콜백의 인자)

`current_motion_id`(지금 재생 중인 번호, 0=없음)가 핵심입니다. `pvector_index` 는 내부 전송 진행일 뿐이라 "끝났는지" 판정에 쓰면 안 됩니다 — 완료는 반드시 결과(`wait()`)로 확인하세요.

상수 · 환경변수

`MIN_MOTION_ID=1` · `MAX_MOTION_ID=50` · `NO_MOTION_ID=0` · `MAX_SEQUENCE_LENGTH=1` · `STATE_FRESH_LIMIT_MS=1500`.

환경변수(`PHORCE_ROBOT_NAMESPACE` 등)는 기본값으로 두면 됩니다 — 운영진이 알려 줄 때만 바꾸세요.

Python 예제 (전부 설치돼 있음)



예제도 실물 로봇에 연결됩니다 — 재생 예제를 실행하면 로봇이 실제로 움직입니다. 실행 전 주변을 확인하세요.

파일	보여주는 것
<code>01_first_motion.py</code>	연결 → 재생 → 결과. 최소형
<code>02_read_feedback.py</code>	1kHz 피드백 구독 (QoS 설정법 포함)과 <code>valid</code> 판정
<code>03_feedback_to_motion.py</code>	보고(1kHz) 와 판단(2Hz) 의 속도 분리 — 여러분 프로젝트의 뼈대로 추천
<code>raw_action.py</code>	SDK 도우미 없이 ROS 2 액션을 직접 부르는 법

프로젝트에 연결하기

```
#include <phorce.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <pthread.h>
#include "phorce_cpp/motion_client.hpp"
using namespace phorce;
// 헤더지 이본은 phorce_cpp 디렉, 클래스는 phorce_ 네임스페이스에 있음
```

제일 중요한 규칙: 이 클라이언트는 스스로 돌지 않습니다. `attach()` 는 여러분 노드에 얹힐 뿐이라서, 여러분이 `rclcpp::spin(node)`(또는 `executor`)을 돌려야 콜백과 결과가 진행됩니다. "아무 일도 안 일어나요"의 원인 1 위가 `spin` 누락입니다.

MotionClient

[illegible]

멤버	뜻
----	---

`attach(node, Target = Target::robot())` 클라이언트 생성 (정적 함수)

`play_async(번호, 콜백 = {})` 재생 시작. 잘못된 번호는 서버에 보내지도 않고 즉시 거절로 완결

`motions_async()` 모션 목록을 future 로

`latest_status()` / `set_status_callback(cb)` 최신 상태 / 상태 콜백

`action_ready()` / `wait_for_action_server()` 서버 준비 확인 / 대기

PlayOperation · PlayResult

멤버	뜻
<code>op->result()</code>	결과 future. 콜백 안에서 <code>.get()</code> 으로 기다리면 멈춥니다(데드락) — <code>wait_for(0ms)</code> 로 "됐나?"만 물어보세요
<code>op->cancel()</code> / <code>op->done()</code>	취소 부탁 (E-Stop 아님) / 끝났는지
<code>r.ok()</code>	성공 판정 (C++ 은 메서드 — Python 과 달리 괄호 있음)
<code>r.busy()</code>	거절 5(바쁨)에서만 true — 이때만 재시도
<code>r.needs_operator()</code>	거절 12·13 — 기다려도 안 풀리니 detail 을 사용자에게 보여 주세요
<code>r.status_name()</code> / <code>r.reject_name()</code>	사람이 읽는 이름

상수: `kMinMotionId=1` · `kMaxMotionId=50` · `kNoMotionId=0` · `kMaxSequenceLength=1` · `kStateFreshLimitMs=1500`.

Status 구조체는 Python 탭의 Status 와 같은 내용이고 판정 규칙(`is_fresh()` · `state_name()`)도 같습니다.

C++ 예제 (전부 설치돼 있음)

실행 명령	보여주는 것
<code>ros2 run phorce_cpp phorce_example_01_first_motion 1</code>	연결 → 재생 → 결과
<code>ros2 run phorce_cpp phorce_example_02_read_feedback</code>	1kHz 피드백 구독
<code>ros2 run phorce_cpp</code> <code>phorce_example_03_feedback_to_motion</code>	타이머로 판단 주기 분리 — 콜백에서 안 기다리는 법
<code>ros2 run phorce_cpp phorce_cpp_async_example 3</code>	단일 executor 논블로킹 전체 흐름
<code>ros2 run phorce_cpp phorce_raw_action_cpp 1</code>	SDK 도우미 없이 액션 직접 호출

ROS 2 직접 — 통신 규약을 그대로 쓰기

SDK 도우미(Python/C++)는 결국 아래 규약을 대신 불러 주는 것뿐입니다. 다른 언어를 쓰거나 기존 ROS 2 노드에 얹고 싶다면 직접 불러도 됩니다.

이름 지도

이름	종류	타입 (agx_msgs)	비고
<code>/</code> <code>motion_action_server/</code> <code>play_motion_sequence</code>	액션	<code>action/</code> <code>PlayMotionSequence</code>	로봇을 움직이는 유일한 창구 1kHz 관절 관측. 구독 QoS 를 반드시 SensorDataQoS(best-effort)로 — 기본값(reliable)으로 구독하면 에러 없이 0 건 옵니다
<code>/phorce/feedback</code>	토픽	<code>msg/PhorceFeedback</code>	
<code>/phorce/status</code>	토픽	<code>msg/PhorceStatus</code>	10Hz 통신 요약 (수신율·모드 등)

`/phorce/motion_window` 토픽 `msg/MotionWindowState` 2Hz, 마지막 값 유지(latched)
`/motion_action_server/` 토픽 `msg/MotionSlotState` 모션 상태 (읽기 전용 — 이걸 폴링해서 발사 타이밍을 재는 루프는 금지)
`motion_slot_state`
`/motion_action_server/` 서비스 `srv/ListMotionSlots` 모션 목록
`list_motion_slots`

```

// PlayMotionSequence
//
// @param motion_ids: motion_ids (배열이지만 원소는 정확히 1 개만 허용됩니다. stop_on_error 는 형식상 있는 필드라
//                    신경 쓰지 않아도 됩니다.)
// @param pvector_index: pvector_index (내부 전송 진행 — 완료 판정 금지, 255=해당 없음) 등.
// @param status: status (0 SUCCEEDED / 1 REJECTED / 2 ABORTED / 3 CANCELED), completed_count,
// @param reject_reason: reject_reason (아래 표), detail (한국어 안내), 그리고 문맥 필드들.

```

PlayMotionSequence 액션의 세 부분

Goal (보내는 것): `motion_ids` — 배열이지만 원소는 정확히 1 개만 허용됩니다. `stop_on_error` 는 형식상 있는 필드라 신경 쓰지 않아도 됩니다.

Feedback (진행 소식): `current_motion_id`(0=재생 없음), `pvector_index`(내부 전송 진행 — 완료 판정 금지, 255=해당 없음) 등.

Result (결과): `status`(0 SUCCEEDED / 1 REJECTED / 2 ABORTED / 3 CANCELED), `completed_count`, `reject_reason`(아래 표), `detail`(한국어 안내), 그리고 문맥 필드들.

거절 코드 전체

코드	이름	뜻
0	NONE	거절 아님
1	EMPTY_SEQUENCE	번호를 안 넣음

- | | | |
|----|-----------------------------------|-----------------------------|
| 2 | <code>SEQUENCE_TOO_LONG</code> | 번호를 2 개 이상 넣음 |
| 3 | <code>MOTION_ID_RANGE</code> | 1~50 밖의 번호 (0 포함) |
| 4 | <code>MOTION_ID_NOT_LOADED</code> | 목록에 없는 번호 |
| 5 | <code>QUEUE_FULL</code> | 다른 모션 재생 중 — 기다리면 풀림 |
| 6 | <code>MASTER_NOT_OP</code> | EtherCAT 버스가 운전(OP) 상태 아님 |
| 7 | <code>COMMAND_SOURCE</code> | 명령 창구가 다른 주인에게 있음 |
| 8 | <code>STATE_STALE</code> | 로봇 상태 정보가 너무 낡음 |
| 9 | <code>SUPERVISOR_VETO</code> | 안전 감시자가 거부 |
| 10 | <code>CONTRACT_NOT_ACTIVE</code> | 모션 창구가 안 열림 |
| 11 | <code>AXIS_NOT_OPERATIONAL</code> | 관절이 운전 상태 아님 |
| 12 | <code>NOT_READY_FOR_MOTION</code> | 영점 버튼 필요 (공통 규칙 4-3) |
| 13 | <code>RECOVERY_REQUIRED</code> | 사람 복구 필요 (공통 규칙 4-3) |

정식 상수 이름에는 `REJECT_` 접두사가 붙습니다 (예: `REJECT_QUEUE_FULL`) — wire 메시지와 Python `.reason` 문자열이 그 형태입니다. 코드 비교는 숫자가 안전합니다.

문맥 필드 — 읽기만 하세요

결과와 상태에 함께 실리는 `boot_id`(로봇의 부팅 세대 번호)와 `request_id`(요청마다 발급되는 번호) 쌍이 "어느 요청 이야기인가"의 신분증입니다. **여러분이 만들지 않습니다** — 시스템이 발급하고, 여러분은 결과 대조에 읽기만 합니다. `physical_idle` 은 "로봇이 물리적으로 멈춰 있다"는 로봇의 직접 확인입니다 — 다른 필드에서 멈춤을 추측하지 마세요. 이 대조를 직접 하기 싫으면 Python/C++ 의 `ok` 를 쓰면 됩니다(그 일을 해 줍니다).

GUI — 화면 프로그램 phorce-console

코드 없이 로봇을 보고 움직여 볼 수 있는 화면입니다. 터미널에 `phorce-console` 이라고 치면 켜집니다.

화면 구성

부분	무엇
상단 배지	연결 상태 · 수신 속도(Hz) · 활성 관절 수 · E-Stop 여부가 한눈에
그래프	관절의 위치·속도·전류·온도 실시간 곡선. 일시정지와 확대 가능
오른쪽 관절 목록	12 개 관절 줄 — 줄을 클릭하면 그 관절이 그래프에 추가됩니다
모션 번호를 골라 버튼으로 재생.	켜자마자 대상 칸을 "● 실물 로봇"으로 바꾸세요 — 테두리가 빨개지고
모션 패널	버튼이 [실물 전송]으로 바뀝니다. (기본값 "시뮬레이터"는 이 행사에서 쓰지 않습니다 — 그대로 두면 로봇이 안 움직입니다.)

실행 옵션

옵션	뜻
----	---

`--no-auto-connect` 관측만 하고 재생 기능은 끄 (그래프 전용으로 쓸 때)

화면의 재생 버튼에는 보호 장치가 있습니다 — 연타해도 0.5 초에 한 번만 나가고, 재생 중에는 새 요청을 만들지 않습니다.

6. 피드백 데이터 사전 — 1kHz 로 오는 것의 전부

`/phorce/feedback` 한 건 = 그 순간의 로봇 스냅샷 한 장. 공통 정보 + 관절 12 칸(`axis[12]` — 현재 기체는 6 칸 사용)으로 구성됩니다.

공통 정보

필드	뜻
stamp / recv_monotonic_ns	발행 시각 / 취득 시각 — 그래프의 x 축이나 시간 분석은 꼭 <code>recv_monotonic_ns</code> 로
wkc	통신 정상 확인 숫자 (정상 3)
tx_cycle_seq	로봇 쪽 사이클 번호 — 건너뛰면 유실이 있었다는 뜻
am_rx_seq_echo / relay_seq_echo	로봇이 마지막으로 받아 준 명령 번호의 메아리
axis_valid_mask 등 4 개 마스크	12 관절의 valid/stale/운전중/결함을 비트로 요약 (bit N = 관절 N)
am_rx_age_ms	로봇이 명령을 못 받은 지 얼마나 됐나 (65535 = 오래됨 고정)
status_flags	로봇 안전 상태 비트

관절 한 칸 (`axis[i]`)

필드	단위	뜻
position_rad	rad	관절 각도
velocity_rad_s	rad/s	관절 속도
current_a	A	모터 전류 — 힘(토크)에 비례합니다
dob_a	A	바깥에서 미는 힘의 추정값 (외란 관측기)
bus_v / temp_c	V / °C	전압 / 온도
pos_ref_echo_rad	rad	관절이 실제로 받은 목표 각도의 메아리
kp_echo / kd_echo	A/rad ·	적용 중인 제어 게인의 메아리

A/(rad/s)

<code>abs_valid</code>	0/1	절대 인코더(전원 꺼도 각도를 기억하는 센서) 정상 여부
<code>loop_cnt</code> / <code>axis_seq</code> / <code>age_ms</code>	—	관절 내부 카운터 / 새 데이터마다 +1 / 데이터 나이(255=한참 됨 · 미수신)
<code>valid</code> · <code>stale</code> · <code>oper</code> · <code>fault</code>	참/거짓	아래 표

믿어도 되나? — 진리표

상황	<code>valid</code>	<code>stale</code>	해석
신선한 데이터 (0~4ms)	참	거짓	믿고 쓰세요
낡은 데이터 (5ms 이상)	거짓	참	판단에 쓰지 마세요
데이터가 아예 없음 (미장착 관절 등)	거짓	거짓	" stale 아니니까 괜찮네 "가 틀리는 이유가 이 줄입니다 — 증거는 <code>valid</code> 하나뿐

7. 함정 모음 — 전원이 한 번씩 밟습니다

함정	증상	바른 길
피드백 구독 QoS 기본값	에러도 없이 0 건 수신	SensorDataQoS(best-effort)로 구독 — 예제 02 참고
1kHz 콜백 안에서 <code>play()</code>	거절 5 폭탄	콜백은 저장만 , 판단·발사는 느린 주기(예: 2Hz)에서 — 예제 03 참고
<code>result.ok()</code> (Python)	에러 또는 오판	Python 은 괄호 없이 <code>result.ok</code> , C++ 만 <code>ok()</code>
거절 12·13 을 "바쁨" 취급	무한 재시도	재시도는 5 만. 12·13 은 사람이 할 차례

C++ 콜백 안에서

`future.get()`

프로그램 멈춤

`wait_for(0ms)` 로 물어만 보기

`pvector_index` 로 완료 판정

영영 안 끝남

완료는 결과(result)로만

`!stale` 을 신선 증거로

빈 관절을 신선으로

오판

증거는 `valid` 하나뿐 (6 장 진리표)

cancel 직후 바로 다음 play 거절 5

cancel 은 즉시 정지가 아님 — **결과를 받고** 다음 요청

8. 보여도 만지면 안 되는 것

토픽 목록을 뒤지다 보면 아래가 보일 수 있습니다. 시스템 내부용이니 직접 부르지 마세요 — 로봇을 움직이는 공식 창구는 `PlayMotionSequence` 액션 하나입니다.

이름	정체
<code>SubmitMotionRequest</code> 서비스	시스템 내부의 전달 통로
<code>/phorce/aperiodic</code> 토픽	정비용 진단 이벤트
<code>PhorceCommand</code> 메시지	내부 명령 형식 — 해커톤 범위 밖
<code>phorce_monitor</code> 실행 설정	로봇 통신 담당 프로그램의 설정 (운영자 영역)

phorce 해커톤 참가자 문서 ④ API 레퍼런스 — 함께 보기: ① 퀵 가이드 · ② 튜토리얼 · ③ 매뉴얼 · ⑤ 시스템 안내